

Feuille de TD 3 : Algorithmes élémentaires sur les graphes

I. Implémentations de graphes

Exercice 1 Étant donnée une représentation par listes d'adjacences d'un graphe orienté, combien de temps faut-il pour calculer le degré sortant de tous les sommets ? Et pour calculer les degrés entrants ?

Exercice 2 Donner une représentation par listes d'adjacences pour une arborescence binaire complet à 7 sommets. Donner la représentation équivalente par matrice d'adjacences. On suppose que les sommets sont numérotés de 1 à 7 comme dans un tas binaire.

Exercice 3 La *transposée* d'un graphe orienté $G = (S, A)$ est le graphe ${}^T G = (S, {}^T A)$, où ${}^T A = \{(v, u) \in S \times S : (u, v) \in A\}$. Autrement dit, ${}^T G$ est obtenu en inversant le sens de tous les arcs de G . Décrire des algorithmes efficaces permettant de calculer ${}^T G$ à partir de G , quand G est représenté par des listes d'adjacences et par une matrice d'adjacences. Analyser le temps d'exécution de vos algorithmes.

Exercice 4 Étant donnée une représentation par listes d'adjacences d'un multigraphe $G = (S, A)$, décrire un algorithme en $O(S + A)$ permettant de calculer la représentation par listes d'adjacences du graphe non orienté $G' = (S, A')$ « équivalent », où A' est constitué des arcs de A , mais avec toutes les arcs multiples entre deux sommets remplacées par une arête unique, et toutes les boucles supprimées.

Exercice 5 Le *carré* d'un graphe orienté $G = (S, A)$ est le graphe $G^2 = (S, A^2)$ tel que $(u, w) \in A^2$ si et seulement s'il existe un $v \in S$, tel que $(u, v) \in A$ et $(v, w) \in A$. Autrement dit, G^2 possède un arc entre u et w chaque fois que G contient un chemin de longueur deux exactement entre u et w . Décrire des algorithmes efficaces permettant de calculer G^2 à partir de G , quand G est représenté par listes d'adjacences, puis par matrice d'adjacences. Analyser le temps d'exécution de vos algorithmes.

Exercice 6 Quand on utilise la représentation par matrice d'adjacences, la plupart des algorithmes de graphes s'exécutent en $O(S^2)$, mais il y a des exceptions. Montrer qu'il est possible de déterminer en $O(S)$ si un graphe orienté contient un *trou noir*, c'est-à-dire un sommet de degré entrant $|S| - 1$ et de degré sortant 0 si on utilise une représentation par matrice d'adjacences.

II. Algorithme de parcours en largeur

Exercice 1 Donner les valeurs d et p d'un parcours en largeur sur le graphe orienté de la figure 22.2(a), en prenant pour origine le sommet 3.

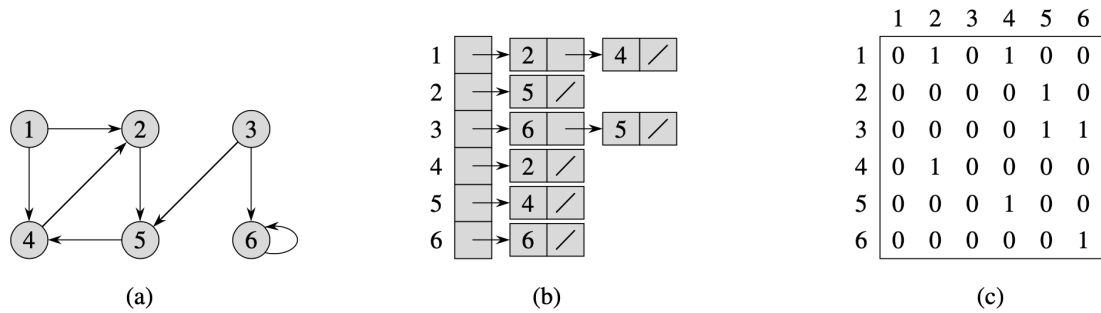


Figure 22.2 Deux représentations d'un graphe orienté. (a) Un graphe orienté G possédant six sommets et huit arcs. (b) Une représentation de G par listes d'adjacences. (c) La représentation de G par matrice d'adjacences.

Exercice 2 Donner les valeurs $d[u]$ qui résultent d'un parcours en largeur d'un graphe non orienté de la figure 22.3, en prenant pour origine le sommet u .

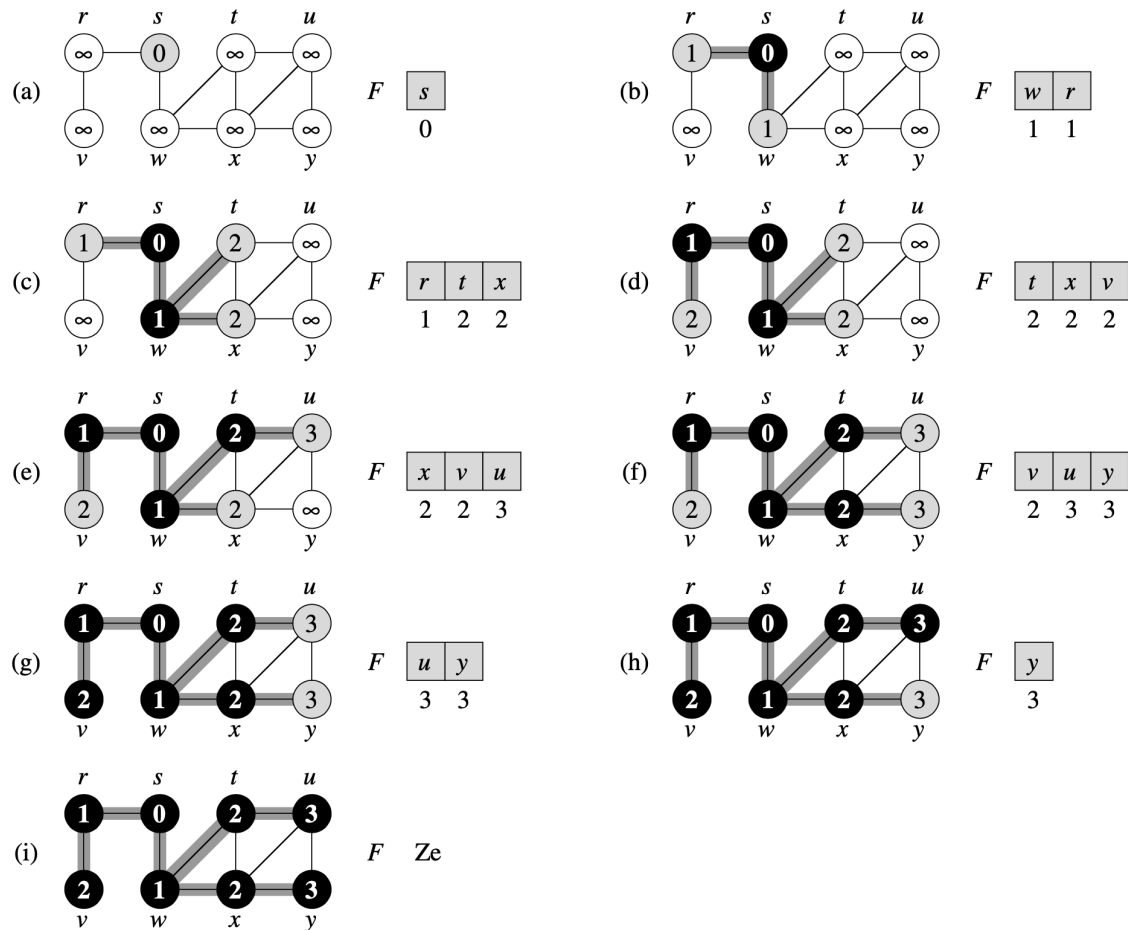


Figure 22.3 L'action de PL sur un graphe non orienté. Les arcs de l'arborescence sont ombrés quand ils sont produits par PL. Les valeurs $d[u]$ sont données à l'intérieur de chaque sommet u . La file F est représentée au début de chaque itération de la boucle **tant que** des lignes 9–18. Les distances à l'origine sont représentées sous chaque sommet de la file.

Exercice 3 Quel est le temps d'exécution de PL si son graphe d'entrée est représenté par une matrice d'adjacences et que l'algorithme est modifié pour gérer ce type d'entrée ?

Exercice 4 Montrer que, dans un parcours en largeur, la valeur $d[u]$ affectée à un sommet u est indépendante de l'ordre dans lequel les sommets apparaissent dans chaque liste d'adjacences. En prenant comme exemple la figure 22.3, montrer que l'arborescence de parcours en largeur calculée par PL peut dépendre de l'ordre au sein des listes d'adjacences.

Exercice 5 Donner un exemple de graphe orienté $G = (S, A)$, de sommet origine $s \in S$ et d'en-semble de trois arcs $A_p \subseteq A$ tels que, pour chaque sommet $v \in S$, le chemin unique dans A_p de s à v soit un plus court chemin dans G , sans que l'ensemble d'arcs A_p puisse être obtenu en exécutant PL sur G , quel que soit l'ordre des sommets dans chaque liste d'adjacences.

Exercice 6 Il existe deux types de catcheurs : les « bons » et les « méchants ». Entre deux catcheurs donnés, il peut ou non y avoir une rivalité. Supposez que l'on ait n catcheurs et une liste de r paires de catcheurs telle qu'il y ait rivalité entre les deux membres de chaque paire. Donner un algorithme à temps $(n + r)$ qui détermine s'il est possible de désigner certains catcheurs comme étant les bons et les autres comme étant les méchants, de telle sorte que chaque rivalité oppose un bon à un méchant. S'il est possible de faire ce genre de classification, votre algorithme doit la calculer.

Exercice 7 Le *diamètre* d'une arborescence $T = (S, A)$ est donné par

$$\max_{u, v \in S} d(u, v) ;$$

$u, v \in S$

autrement dit, le diamètre est la plus grande de toutes les distances de plus court chemin dans l'arborescence. Donner un algorithme efficace permettant de calculer le diamètre d'une arborescence, et analyser son temps d'exécution.

Exercice 8 Soit $G = (S, A)$ un graphe connexe, non orienté. Donner un algorithme en $O(S + A)$ permettant de calculer une chaîne de G qui traverse chaque arête de A exactement une fois dans chaque direction. Décrire une manière de trouver son chemin dans un labyrinthe quand on dispose d'une grande quantité de pièces de monnaie.

III. parcours en profondeur

Exercice 1 Dessiner un tableau 3×3 en étiquetant les lignes et les colonnes avec BLANC, GRIS et NOIR. Dans chaque cellule (i, j) , indiquer s'il peut exister, pendant le parcours en profondeur d'un graphe orienté, un arc reliant un sommet de couleur i à un sommet de couleur j . Pour chaque arc possible, indiquer de quel(s) type(s) il peut être. Dessiner un second tableau pour le parcours en profondeur d'un graphe non orienté.

Exercice 2 Donner le déroulement du parcours en profondeur sur le graphe de la figure 22.6. On supposera que la boucle **pour** des lignes 5–7 de la procédure PP considère les sommets dans l'ordre alphabétique, et que chaque liste d'adjacences est ordonnée

alphabétiquement. Donner les dates de découverte et de fin de traitement de chaque sommet, ainsi que la classification de chaque arc.

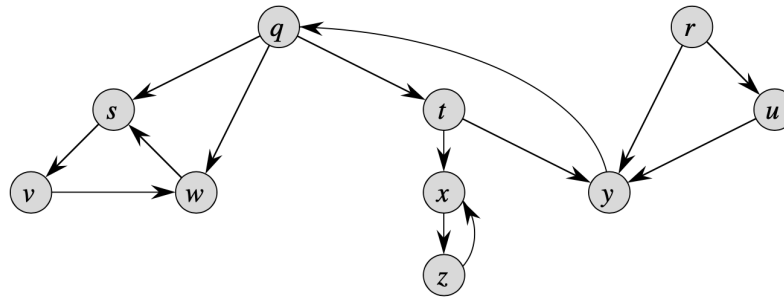


Figure 22.6 Un graphe orienté servant de support aux exercices 22.3.2 et 22.5.2.

Exercice 3 Mettre en évidence la structure parenthésée du parcours en profondeur illustré à la figure 22.4.

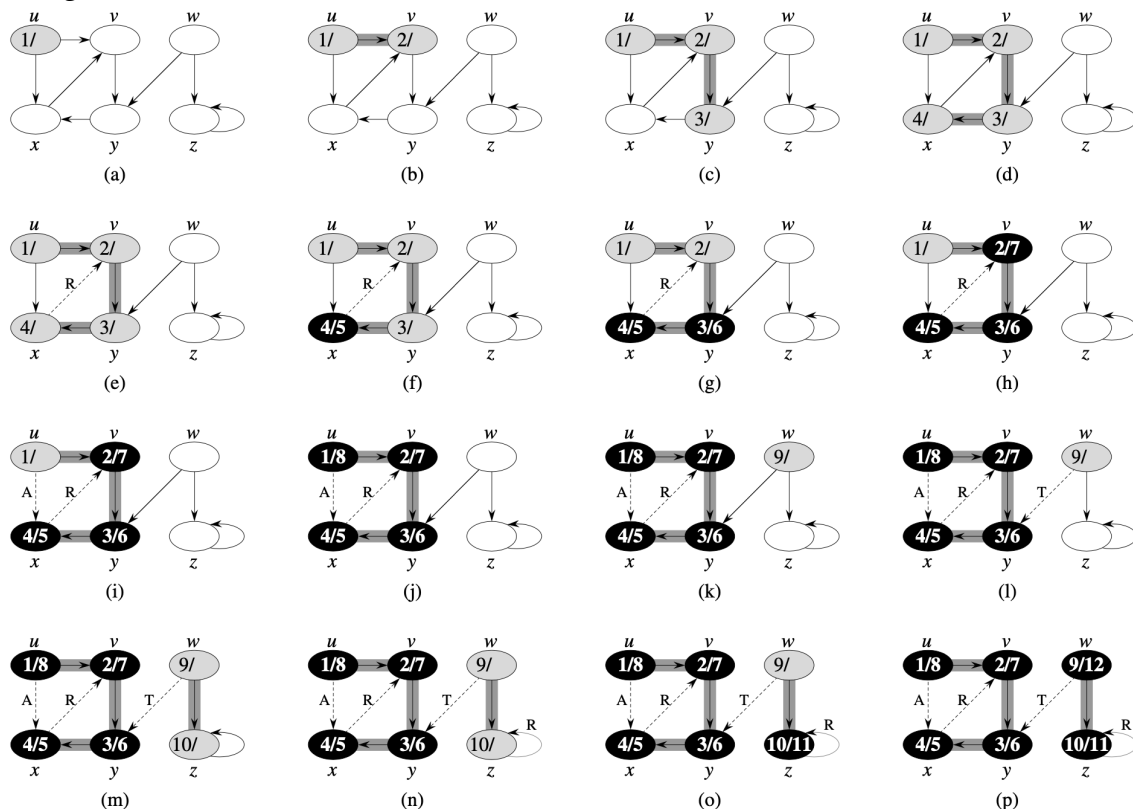


Figure 22.4 La progression de l'algorithme de recherche en profondeur PP sur un graphe orienté. Au fur et à mesure que les arcs sont explorés, ils sont représentés sur fond ombré (si ce sont des arcs de liaison) ou tracés en pointillés (sinon). Les arcs ne faisant pas partie des arborescences (arcs qui ne sont pas des arcs de liaison) sont étiquetés R, T ou A selon que ce sont des arcs arrières, transverses ou avant. Les sommets sont étiquetés par leur dates de découverte et de fin de traitement.

Exercice 4 Réécrire PP, en utilisant une pile pour supprimer la récursivité.

Exercice 5 Donner un contre-exemple de la conjecture selon laquelle s'il existe un chemin entre u et v dans un graphe orienté G , et si $d[u] < d[v]$ lors d'un parcours en profondeur de G , alors v est un descendant de u dans la forêt de parcours en profondeur obtenue.

Exercice 6 Donner un contre-exemple de la conjecture selon laquelle s'il existe un chemin entre u et v dans un graphe orienté G , alors tout parcours en profondeur donne forcément $d[v] \leq f[u]$.

Exercice 7 Montrer qu'un parcours en profondeur d'un graphe non orienté G peut servir à identifier les composantes connexes de G , et que la forêt de parcours en profondeur contient autant d'arborescences que G possède de composantes connexes. Plus précisément, montrer comment modifier le parcours en profondeur pour que chaque sommet v soit affecté d'une étiquette entière $cc[v]$ comprise entre 1 et k , où k est le nombre de composantes connexes de G , telle que $cc[u] = cc[v]$ si et seulement si u et v appartiennent à la même composante connexe.